Dynamic JSON-Based Configuration Renderer (Frontend)

Context

Many internal platforms generate UI dynamically from **backend-provided configuration** rather than hardcoded forms. These configurations are often **deeply nested**, evolve over time, and are shared across multiple systems.

We want to evaluate your ability to:

- Interpret JSON configuration
- Render UI **entirely from data**
- Manage state cleanly for **dynamic and nested structures**
- Build something that can scale beyond a toy example

This assignment focuses on **rendering and editing configuration**, not styling or visual polish (barebones CSS should work so that the UX is clear and understandable)

Problem Statement

You are given a JSON configuration object returned by a backend system.

Your task is to build a **React component** that:

- Intelligently create a ‘typed’ JSON schema from the JSON config API output
- Dynamically renders editable form fields based on the ‘typed' JSON schema that was created in the previous step
- Allows users to modify values
- Produces the updated configuration on submit

The UI must be generated **entirely from the JSON**, with **no hardcoded JSX for specific fields**.

Sample Configuration (Initial Version)

```
{
  "cittaAgent": {
    "affinity": {
      "affinityType": "capabilitySubset"
    },
    "dataBucket": {
      "bucketName": "alpha.tenant.data",
      "rootPath": "qinstance1-tenant1",
      "s3IAMRole": "citta_emr_serverless_role"
    }
  },
  "cittaAgentSet": {
    "default": {
      "cittaAgent": {
        "affinity": {
          "label": "default"
        },
        "swlhType": "AWS_EMR_SERVERLESS_BACKED"
      }
    },
    "default_new": {
      "cittaAgent": {
        "affinity": {
          "label": "default_new"
```

```
        },
        "swlhType": "AWS_EMR_SERVERLESS_BACKED"
      }
    }
  }
}

```

This is a **representative configuration**, not a fixed schema.

Functional Requirements

1. Dynamic Rendering

- Render editable form inputs based on the JSON structure
- Support at minimum:
  - Strings → text inputs
  - Numbers → number inputs
  - Booleans → checkboxes
  - Arrays
- Nested objects must be rendered hierarchically (sections / groups)

2. State Management

- Maintain form state dynamically
- Changes to any field must update the corresponding location in the JSON
- Do not flatten the structure

3. Submission

- On submit:
  - Log the **final updated JSON**
  - Preserve the original structure

4. No Hardcoding

- No JSX tied to specific keys like `bucketName`, `affinityType`, etc.
- The solution should work for **any similarly shaped config**

Non-Functional Requirements

- Clean, readable logic
- Clear separation of concerns
- Reasonable component structure
- You can use form-based libraries but CANNOT use the feature to convert a JSON into a form directly

---

What We Are Looking For

- Ability to reason about **data-driven UI**
- Clean dynamic state handling
- Comfort with recursive rendering

- Thoughtful component design
- Readable, maintainable code

Bonus (Optional)

- Collapsible sections for nested objects
- Read-only vs editable mode
- Field path display (e.g. `cittaAgent.dataBucket.bucketName`)
- Support for arrays